

Video 1.2: Container vs. VM Security Models

Duration: 7:00

Format: PPT (35%) / Terminal (35%) / Browser (20%) / Code Editor (10%)

Resolution: 1920x1080 @ 30fps

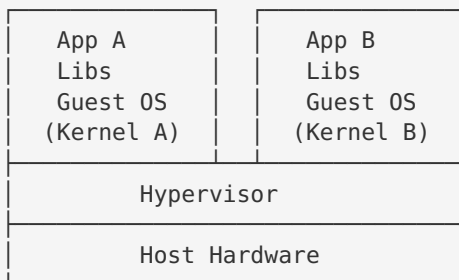
Prerequisites: Video 1.1 (Namespaces, cgroups basics)

SCRIPT BEGIN

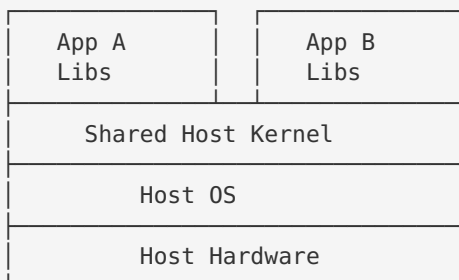
[0:00-0:50] — The Fundamental Difference (PPT Slide)

SLIDE: Side-by-side architecture diagram:

Left — Virtual Machine:



Right — Container:



NARRATION:

“This is the diagram you’ve probably seen a hundred times. But let’s read it as a security engineer. On the left, virtual machines — each VM has its own kernel, its own operating system, running on top of a hypervisor. The hypervisor mediates all access to hardware. On the right, containers — they share the host kernel directly. There is no hypervisor. There is no guest OS. Every container’s system calls go straight to the same kernel. This is the fundamental architectural difference, and from a security perspective, it changes everything.”

[0:50-1:45] — Attack Surface Comparison (PPT Slide)

SLIDE: Title: "Attack Surface Comparison"

Dimension	Virtual Machine	Container
Kernel exposure	Guest kernel only	Shared host kernel
Escape complexity	Hypervisor exploit (rare)	Kernel exploit (more common)
Syscall surface	Hypervisor-mediated	Direct to host kernel
Network isolation	Virtual NIC, separate stack	Virtual bridge, shared kernel stack
Filesystem	Virtual disk image	Layered overlay on host FS
Blast radius of escape	Access to hypervisor	Access to host + ALL containers

NARRATION:

"Let me break this down. In a VM, if an attacker compromises the guest OS, they're still inside a virtualized environment. To reach the host, they need a hypervisor escape — these exist, but they're rare and extremely valuable. In a container, if an attacker compromises the containerized application, they're one kernel vulnerability away from the host. The system call surface goes directly to the host kernel — there's no translation layer. And if they escape? They don't just get the host — they potentially get access to every container running on that host. The blast radius is fundamentally larger."

[1:45-3:15] — Proving the Shared Kernel (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

"Let me prove this is real, not theoretical."

COMMAND + EXPLANATION:

```
# Show host kernel version
$ uname -r
5.15.0-92-generic
```

"Here's the host kernel version."

```
# Run containers with different "OS" images
$ docker run --rm alpine uname -r
5.15.0-92-generic

$ docker run --rm ubuntu:22.04 uname -r
5.15.0-92-generic

$ docker run --rm centos:7 uname -r
5.15.0-92-generic
```

“Now watch — I’m running Alpine, Ubuntu, and CentOS containers. Every single one reports the same kernel version. They’re all running on the host kernel. The container image provides the userspace — libraries, package managers, shell — but the kernel is shared. Always.”

```
# Show that a container can see host kernel messages (without filtering)
$ docker run --rm --privileged alpine dmesg | tail -5
```

“With privileged access, a container can even read the host kernel’s message buffer. This is the shared kernel in action.”

```
# Show shared kernel modules
$ docker run --rm --privileged alpine lsmod | head -10
```

“Same kernel modules too. Every kernel module loaded on the host is accessible from inside the container. If a kernel module has a vulnerability, every container on that host is exposed.”

[3:15-4:15] — The /proc and /sys Exposure (Terminal)

NARRATION:

“Let’s look at another critical difference — what the container can see about the host through pseudo-file systems.”

COMMAND + EXPLANATION:

```
# Default container – some host info leaks through
$ docker run --rm alpine cat /proc/version
Linux version 5.15.0-92-generic ...
```

“The container can read the host’s kernel version string from /proc/version.”

```
# Host CPU info is visible
$ docker run --rm alpine cat /proc/cpuinfo | head -20
```

“It can see the exact CPU model, number of cores, and processor flags. This is information leakage — an attacker uses this for reconnaissance.”

```
# Memory info is partially visible
$ docker run --rm alpine cat /proc/meminfo | head -5
```

“And it can see the host’s total memory — not just the container’s cgroup limit. Without the right hardening, containers leak host information by default.”

```
# Compare with a hardened container
$ docker run --rm --read-only \
  --security-opt=no-new-privileges \
  --cap-drop=ALL \
  alpine cat /proc/cpuinfo | head -5
```

“Even with common hardening flags, this information still leaks. We need additional measures like read-only /proc mounts or tools like Sysbox to prevent this. We’ll cover hardening in depth in later modules.”

[4:15-5:15] — VM Escape vs. Container Escape (PPT + Browser)

SLIDE: Title: “Escape Complexity Comparison”

VM Escape (CVE-2015-3456 — VENOM):

- Required exploiting a flaw in QEMU’s virtual floppy disk controller
- Affected the hypervisor hardware emulation layer
- Complexity: Very high — required deep knowledge of hardware emulation
- Impact: Access to hypervisor, then to other VMs

Container Escape (CVE-2019-5736 — runc):

- Exploited a flaw in the container runtime’s file handling
- Overwrote the host’s runc binary from inside the container
- Complexity: Moderate — a crafted container image was sufficient
- Impact: Root access on the host immediately

NARRATION:

“Let’s compare two real escapes. VENOM — a VM escape from 2015 — required exploiting a bug in QEMU’s virtual floppy disk controller. You needed deep knowledge of hardware emulation, and even then, you landed in the hypervisor, not directly on the host OS. Contrast that with CVE-2019-5736 — a container escape from 2019. An attacker could craft a malicious container image that, when run, would overwrite the host’s runc binary. The result? Root code execution on the host. The complexity was moderate — a PoC was publicly available within weeks. This difference in escape complexity is why the container vs. VM security boundary discussion matters.”

SWITCH TO BROWSER:

“Let me show you the advisory.”

BROWSER: Navigate to <https://nvd.nist.gov/vuln/detail/CVE-2019-5736>

“Here on the NVD page — CVSS score of 8.6. The attack vector is local, but in container environments, ‘local’ means any container running on your infrastructure. The description clearly states: allows overwriting the host runc binary and thus gaining host root access.”

[5:15-6:15] — When to Use Containers vs. VMs (PPT Slide)

SLIDE: Title: “Security Decision Matrix”

Scenario	Recommendation	Rationale
Multi-tenant workloads	VM (or VM + container)	Hard isolation between tenants
Running untrusted code	VM or gVisor/Kata	Container isolation insufficient
Microservices (trusted)	Container + hardening	Performance + acceptable risk
CI/CD build pipelines	Ephemeral VMs or sandboxed containers	Build processes are high-risk
Development environments	Containers fine	Risk tolerance higher
Regulatory compliance (PCI, HIPAA)	VM boundary often required	Auditors expect hardware isolation

NARRATION:

“So when should you use containers and when should you insist on VMs? For multi-tenant workloads — where different customers or trust domains share infrastructure — you need VM-level isolation, period. Running untrusted code? Use VMs or specialized runtimes like gVisor or Kata Containers that add a kernel boundary. For trusted microservices with proper hardening, containers are appropriate — the performance benefits are real. CI/CD pipelines are high-risk — build processes pull arbitrary code from the internet and execute it. Use ephemeral VMs or heavily sandboxed containers. And for regulatory compliance — especially PCI-DSS and HIPAA — auditors often require demonstrable hardware-level isolation between security zones.”

[6:15–7:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: “Key Takeaways”

1. Containers share the host kernel — this is an architectural fact, not a bug
2. The attack surface of containers is fundamentally larger than VMs
3. Container escapes are more frequent and less complex than VM escapes
4. Defense-in-depth is mandatory — containers alone are not a security boundary
5. Choose isolation technology based on trust boundaries, not convenience

NARRATION:

“Let me be clear — containers are not insecure by design. They were designed for performance and portability, not security isolation. The shared kernel is a feature for efficiency and a risk for security. Your job as a security professional is to understand this tradeoff and apply defense-in-depth. In the next lesson, we’re going to go deeper into the shared kernel problem — examining exactly which system calls cross the isolation boundary and what that means for your threat model.”

SLIDE: “Next: The Shared Kernel Problem →”

SCRIPT END

Post-Production Notes

- Architecture diagrams should be animated — build from bottom (hardware) up
- For the uname -r demos, use three separate terminal panes tiled horizontally to show all results simultaneously
- The NVD browser page should be pre-loaded; do not show live page loading
- Decision matrix table should highlight the “VM” recommendations in amber/orange
- Add a subtle visual indicator (red border) around the “Container” architecture diagram when discussing security limitations
- Ensure CentOS image is pre-pulled to avoid download delays during recording